

Activitate: metode utile ale listelor în Python — extend(), reverse(), clear(), remove(), sort(), copy()

Metode utile pentru liste în Python

Noțiuni teoretice

La lecțiile anterioare ați lucrat deja cu liste și cu primele lor metode: `append()` pentru adăugarea unui element la final, `insert()` pentru inserarea pe o poziție anume și `pop()` pentru eliminarea unui element după poziție. Clasa `list` mai pune însă la dispoziție și alte metode, foarte des folosite atunci când prelucrăm colecții de date.

`extend()` — adăugarea mai multor elemente deodată

Metoda `extend()` adaugă la finalul unei liste toate elementele altei liste (sau ale oricărei alte colecții), unul câte unul. Se deosebește de `append()`, care ar adăuga întreaga listă ca un singur element.

```
fructe = ["mar", "para"]
fructe.extend(["kiwi", "banana"])
print(fructe)
# ["mar", "para", "kiwi", "banana"]
```

`reverse()` — inversarea ordinii elementelor

Metoda `reverse()` schimbă ordinea elementelor din listă, punând ultimul element primul, penultimul al doilea și așa mai departe. Modificarea se face direct asupra listei, nu se creează o listă nouă.

```
numere = [1, 2, 3, 4, 5]
numere.reverse()
print(numere)
# [5, 4, 3, 2, 1]
```

`clear()` — golirea listei

Metoda `clear()` elimină toate elementele unei liste, aceasta rămânând o listă goală, dar existentă (spre deosebire de a atribui direct `lista_var = []`, lista rămâne același obiect din memorie, doar golit de elemente).

```
note = [8, 9, 10, 7]
note.clear()
print(note)
# []
```

`remove()` — eliminarea unui element după valoare

Metoda `remove(valoare)` caută în listă prima apariție a valorii date și o elimină. Dacă valoarea nu se găsește în listă, Python oprește programul cu o eroare de tip `ValueError`, așa că, în practică, e bine ca înainte de eliminare să verificăm dacă valoarea chiar există în listă.

```
culori = ["rosu", "verde", "albastru", "verde"]
culori.remove("verde")
print(culori)
# ["rosu", "albastru", "verde"]
# a fost eliminata doar prima aparitie a lui "verde"
```

`sort()` — sortarea elementelor

Metoda `sort()` ordonează elementele listei crescător (implicit). Pentru ordine descrescătoare, se folosește parametrul `reverse=True`. Ca și `reverse()`, modificarea se face direct pe listă.

```
varste = [23, 19, 31, 27, 18]
varste.sort()
```

```
print(varste)
# [18, 19, 23, 27, 31]

varste.sort(reverse=True)
print(varste)
# [31, 27, 23, 19, 18]
```

copy() — copierea unei liste

Metoda `copy()` returnează o listă nouă, cu aceleași elemente ca lista inițială. Este utilă atunci când vrem să păstrăm lista originală neschimbată și să facem modificări doar pe copie.

```
lista_a = [1, 2, 3]
lista_b = lista_a.copy()
lista_b.append(4)
print(lista_a)    # [1, 2, 3]
print(lista_b)    # [1, 2, 3, 4]
```

Tabel recapitulativ

Pentru fixarea celor șase metode, completați (sau verificați) în tabelul de mai jos acțiunea principală pe care fiecare metodă o are asupra listei pe care este apelată:

Metodă	Ce face asupra listei
<code>lista.extend(alta_lista)</code>	adaugă la finalul listei, pe rând, toate elementele din <code>alta_lista</code>
<code>lista.reverse()</code>	inversează ordinea elementelor din listă, pe loc, fără să returneze o listă nouă
<code>lista.clear()</code>	șterge toate elementele din listă, aceasta rămânând o listă goală <code>[]</code>
<code>lista.remove(valoare)</code>	elimină din listă prima apariție a elementului cu valoarea dată
<code>lista.sort()</code>	sortează elementele listei în ordine crescătoare (sau după o cheie dată), pe loc
<code>lista.copy()</code>	returnează o listă nouă, o copie a listei inițiale, la aceleași elemente

Exerciții rezolvate

Exercițiul 1 — Gestionarea unei liste de sarcini

Enunț: Pornind de la o listă de sarcini de făcut, adăugați alte două sarcini cu `extend()`, eliminați o sarcină care a fost deja rezolvată, apoi afișați lista sortată alfabetic.

Rezolvare:

```
sarcini = ["teme", "cumparaturi", "antrenament"]
sarcini.extend(["curatenie", "lectura"])
print(sarcini)

sarcini.remove("cumparaturi")
print(sarcini)

sarcini.sort()
print(sarcini)
```

Explicație: `extend()` adaugă cele două sarcini noi la finalul listei. `remove()` scoate din listă sarcina rezolvată, căutând-o după valoare, nu după poziție. `sort()` rearanjează, la final, toate sarcinile rămase în ordine alfabetică.

Exercițiul 2 — Copie de siguranță înainte de modificări

Enunț: Aveți o listă cu punctajele obținute de un elev la teste. Faceți o copie a listei, inversați ordinea elementelor din copie, apoi goliți lista originală, arătând că cele două liste sunt independente una de alta.

Rezolvare:

```
punctaje = [7, 8, 10, 6, 9]
copie = punctaje.copy()

copie.reverse()
print("Copie inversata:", copie)

punctaje.clear()
print("Lista originala:", punctaje)
print("Copia a ramas neschimbata:", copie)
```

Explicație: copy() creează o listă separată, așa că modificările făcute pe una dintre liste (reverse() pe copie, clear() pe original) nu se transmit și celeilalte.

Exerciții propuse

Combinarea a două liste — aveți listele elevi_clasa_a și elevi_clasa_b; folosind extend(), obțineți o singură listă cu toți elevii. (Dificultate: ușor)

Cel mai mic și cel mai mare — dintr-o listă de numere citite de la tastatură, folosiți sort() pentru a afișa cea mai mică și cea mai mare valoare, fără să folosiți funcțiile min() și max(). (Dificultate: ușor)

Eliminarea unui produs din stoc — verificați, cu operatorul in, dacă un produs introdus de utilizator există în lista de produse, iar dacă da, eliminați-l cu remove(); dacă nu există, afișați un mesaj corespunzător. (Dificultate: mediu)

Listă originală vs. listă inversată — porniți de la o listă de cuvinte, faceți o copie cu copy(), inversați copia cu reverse() și afișați ambele liste, una sub alta, pentru a observa că lista originală nu s-a schimbat. (Dificultate: mediu)

Resetarea unei liste de scoruri — scrieți un program cu un mic meniu (adăugare scor, afișare scoruri sortate, resetare listă) care folosește, după caz, metodele append(), sort() și clear(), repetând meniul până când utilizatorul alege să iasă. (Dificultate: dificil)

Exercițiu bonus

Scrieți un program care administrează o listă de invitați la o petrecere. Programul trebuie să permită adăugarea mai multor invitați deodată dintr-o altă listă (extend()), ștergerea unui invitat care a anulat (remove()), afișarea listei în ordine alfabetică (sort()) și, la final, salvarea unei copii a listei finale de invitați (copy()) înainte de a goli lista principală (clear()), pentru a putea compara cele două liste. (Dificultate: avansat)